

C++ in Quantitative Finance II

Project Report

Antoni Smolski (473975)

Objective

This project prices a long put (up-and-out) butterfly that is set up with European barrier options and investigates how the price of the whole portfolio reacts to the spot price, volatility and level of the barrier. I price the options with Monte Carlo in C++ and then call that C++ code from Excel through the xlw wrapper, so the tables and charts in the workbook are produced directly by my pricer.

Assumptions

I assume the underlying follows a geometric Brownian motion under the risk-neutral measure:

$$dS = (r - d)Sdt + \sigma SdW$$

The rate, volatility and the dividend are constant, although the code does not force this. Each of them is stored as a Parameters curve, so a full term structure could be plugged in later without changing the engine easily.

The barrier is monitored continuously. In the code, I created the BarrierMonitor class, which enables discrete monitoring (as in the classes) and Brownian Bridge method as an extra. I was wondering how it would affect the solution and it is used in practise for barrier options. Brownian Bridge adds the probability that the path crossed the barrier somewhere between two grid points. The formula is as follows:

$$q_i = 1 - \exp\left(-\frac{2 \ln\left(\frac{U}{S_i}\right) \ln\left(\frac{U}{S_{i+1}}\right)}{v_i}\right)$$

Where $q_i = 0$ if either endpoint is already at or above U . Here v_i is the variance of the log-return over the step, $v_i = \int_{t_i}^{t_{i+1}} \sigma^2 du = \sigma^2 \Delta t$ for constant volatility, with $\Delta t = T/N$.

The survival probability of the whole path is the product over all N steps and the price is the discounted payoff weighted by it, averaged over the M Monte Carlo paths:

$$\hat{P}_{\text{surv}} = \prod_{i=0}^{N-1} q_i$$
$$\hat{V} = e^{-rT} \frac{1}{M} \sum_{j=1}^M \hat{P}_{\text{surv}}^{(j)} g(S_T^{(j)})$$

where g is the vanilla payoff, e.g. $g(S_T) = \max(K - S_T, 0)$ for the put.

I tested the Brownian Bridge solution separately and it is almost independent of the number of steps. Because the barrier in this problem sits well above the spot (130 against a spot of 110), the discrete result on a reasonably fine grid and the bridge result are very close, so the choice does not change the prices in the report in any meaningful way.

The Monte Carlo settings used by the Excel function are 100 time steps and 100000 paths with antithetic variates and a Mersenne-Twister generator seeded with a fixed number, so the results are reproducible. The antithetic draws come in negatively correlated pairs, so the standard error reported by the simple statistics gatherer, which assumes independent draws, is only approximate. Also, I am aware that we used Park Miller generator and the code contains it and is able to use it. The Park Miller vs Mersenne-Twister is compared in one of the testing scripts.

The base parameters from the assignment are:

$$S_0 = 110$$

$$\sigma = 23\%$$

$$r = 5\%$$

$$d = 0\%$$

$$T = 6M$$

The Portfolio

The long put butterfly is made of four options. They are all puts and all up-and-out as required for puts and they share the same maturity $T = 0.5$:

- long one out-of-the-money put, strike $K_1 = 100$
- short two at-the-money puts, strike $K_2 = 110$
- long one in-the-money put, strike $K_3 = 120$

The strikes were not given, so I put the middle one at the spot and spread the wings symmetrically around it, which is the standard long butterfly. There is a single barrier sitting above the structure (130). I do not build a special butterfly payoff. The value of the portfolio is just the sum of the four option prices with weights +1, -2 and +1 and that is exactly what the Portfolio class does. At the base parameters the butterfly is worth about 1.86.

Code

The code keeps an option and the way it is priced separate, in the same spirit as QuantLib. An instrument such as a BarrierOption holds a payoff and is handed a pricing engine, so the same option can be priced analytically or by Monte Carlo just by swapping the engine.

The payoff is a small functor for a call or a put. The instrument stores the contract and leaves the actual pricing to whatever engine is attached to it.

BarrierOption are named with makers (upOut, downOut etc.) for simplicity. I did this, because I wanted to have one general class for the barrier options and as the double barrier is generalized single barrier I decided to implement double barrier class. However, in order to price single barrier one would need to pass "inf" as one of the barrier in double barrier class. It would be not elegant and easy to forget when resuing the code in the future. Because of that I decided to use the makers.

Portfolio, which is a weighted sum of instruments. The butterfly is simply four legs with weights +1, -2 and +1.

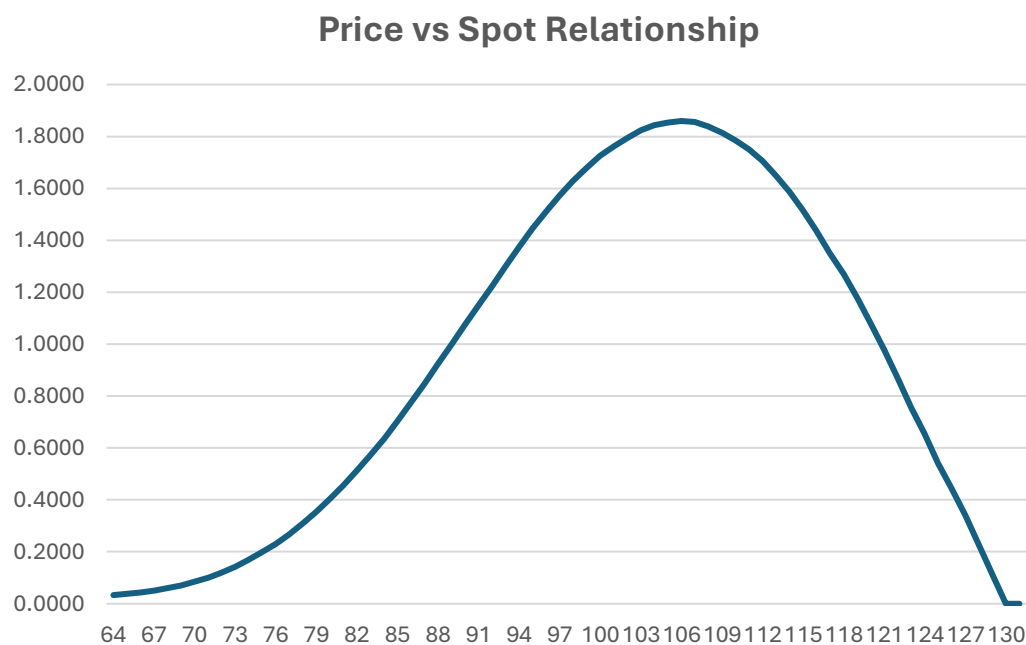
MonteCarloBarrierEngine simulates each path, applies the barrier monitoring and collects the discounted payoff.

The random numbers sit behind a small interface (RngBase). I have a Mersenne-Twister generator and class-developed Park-Miller generator behind that same interface. I decided to do it like this just to practise. On C++ in QF I, we used the Mersenne-Twister and I wanted to have both. As we were using ParkMiller in the classes, I used here the other one for the contrast. I have also an antithetic decorator that can wrap either of them for variance reduction.

Barrier monitoring is also swappable, so a discrete monitor and the Brownian-bridge monitor can be chosen when the engine is built. A statistics gatherer (ConfidenceLimits) keeps the mean, the standard error and a 95% confidence interval (or different if defined by user).

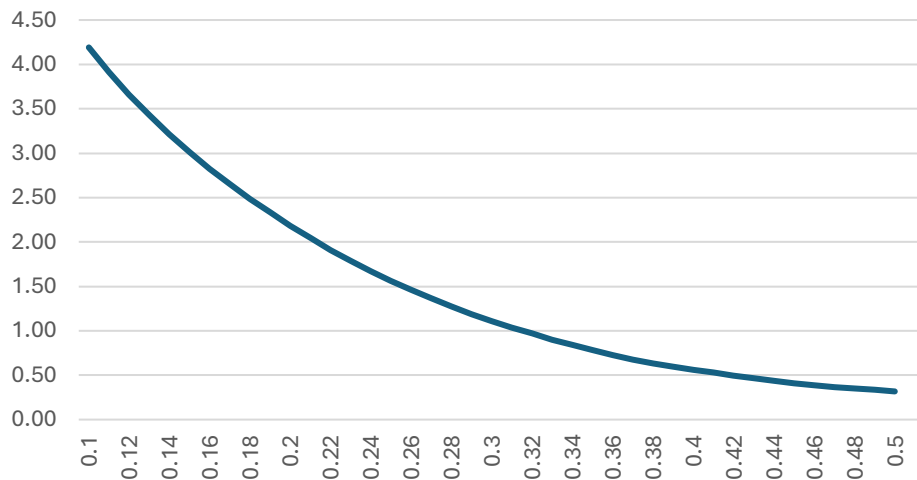
Xlw exposes one function to Excel, PriceButterfly(spot, vol, barrier, expiry, K1, K2, K3, r, d), which builds the butterfly and returns its price. Everything in the workbook is produced by calling this function. I decided to not include the rate and dividend yield just for simplicity. It could easily be provided there as well.

Results



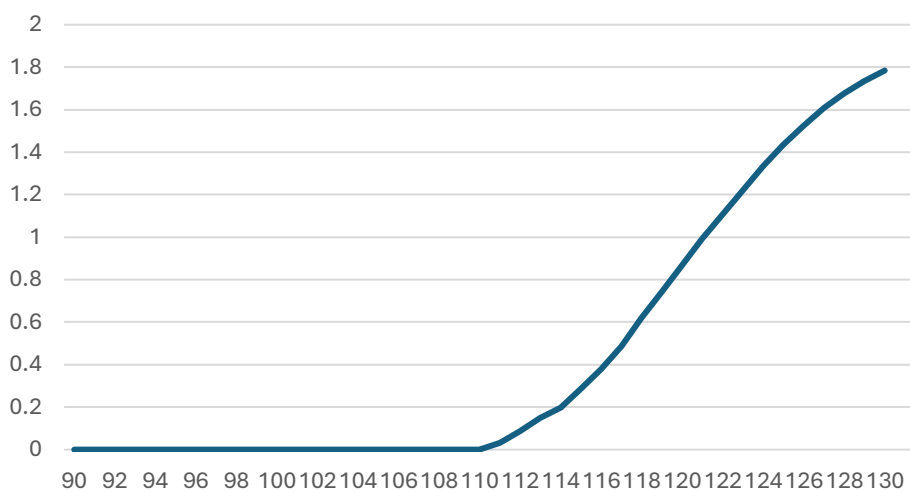
As the spot moves, the price climbs to a peak of about 1.93 around a spot of 105 to 106 and then falls back to zero as the spot gets close to the barrier at 130. The peak sits a little to the left of the middle strike, because the up-and-out feature decreases the value dramatically near higher spot prices as a higher spot is more likely to knock the options out before maturity.

Price vs Volatility Relationship



The price falls steadily as volatility rises, from about 4.24 at a 10% volatility down to about 0.33 at 50%, with no turning point over the range I looked at. A long butterfly is basically a bet that the spot finishes near the middle strike, so it loses value when higher volatility spreads the terminal distribution out. The up-and-out barrier pushes in the same direction, because higher volatility also makes a knock-out more likely, so the two effects reinforce each other.

Price vs Barrier Relationship



While the barrier is at or below the spot then the price is zero. This is because the up-and-out options are knocked out from the very start and the whole structure is worthless. Once the barrier moves above the spot the price rises as there is less chance of being knocked out and it approaches the value of the plain butterfly with no barrier as the barrier moves far away.

Putting the three together, the barrier is what ties the other two factors to the payoff. A barrier far above the spot makes the structure behave almost like an ordinary butterfly, a barrier near the spot kills it and in between the volatility matters more because it controls how easily the path reaches the barrier.

On Monte Carlo accuracy: I checked convergence on a single up-and-out put and the standard error falls like one over the square root of the number of paths, from roughly 0.27 at about a

thousand paths down to roughly 0.008 at a million. In a separate test the Brownian-bridge monitor was almost flat in the number of steps, while the discrete monitor only loses its bias slowly, staying a few percent off even at several hundred steps when the barrier is close to the spot. That is why the bridge is worth having when the barrier is near the money.

Declaration

In accordance with the Honor Code, I certify that my answers here are my own work based on class materials. I did not make my solutions available to anyone else, and that I did not engage in any activities that would dishonestly improve my results or improve or harm the results of others.

A note on the design and on AI use

This project extends the code we built in class so that it does what the assignment asks, but I refactored and partly redesigned it along the way. I wanted to reimplement the classes and redo the class project in a layout that is more convenient for me and that I can keep using later, which is why the design, the folder structure and some of the solutions are different.

I used the ideas from class but made the code more modular. An option is now priced through a payoff plus an engine plus a process, with a few optional pieces such as the monitoring (discrete or Brownian bridge) and the time settings. This makes it easy to swap one part out, for example to change the process from plain GBM to some stochastic-volatility model and the same process could then be fed either into Monte Carlo or into a 1D or 2D PDE solver, which I have not included here. The butterfly itself is priced through the Portfolio class. I did not want to add too many special structures, because a butterfly, or anything similar, is just a portfolio of options and this class wraps them up and prices them through the engines attached to each option. The alternative would be a dedicated butterfly class that prices the whole thing in one Monte Carlo run, but I preferred to keep the legs independent. There is just one helper function "PriceButterfly" implemented, which was created for the purpose of Excel solution.

On AI tools: I used it while learning the concepts, to go through the class repository with me and to help plan this project given what I wanted to achieve. The code is written completely by me with some help when I could not debug it or design a sensible solution.